A REUSABLE PATTERN · LANGUAGE- AND PROJECT-AGNOSTIC

The Agent Development Harness

Seven controls that let a team ship at high throughput with AI agents writing most of the code — and the reasoning behind each, so you can adapt them rather than copy them blindly.

◆ THE PROBLEM THIS SOLVES

When an agent writes the code *and* its tests, a green suite proves almost nothing: if it misread the requirement, both are wrong in the same direction and agree perfectly. Ordinary quality practice assumes an independent human author somewhere in the loop. Remove that assumption and most of it stops working. **Every control here restores one specific piece of independence that automation took away.**

AUDIENCE	ASSUMES	COMPANION
Engineering leads · platform teams	**Git · CI · agents writing code**	**Adoption Guide**

00 Contents

The seven controls, in one table

CONTROL	THE SPECIFIC FAILURE IT REMOVES
Four exit codes	A check that never ran reporting as a pass.
Gate fault suites	A gate that cannot fail, which looks identical to a codebase that is always right.
Decision register	A guess in the code becoming indistinguishable from a decision.
Trigger gates	"Not applicable" silently becoming "unmeasured".
Drift protection	Copied shared files rotting with nothing noticing.
CI ordering laws	One check's failure hiding another's verdict.
Independence controls	An author's tests confirming the author's misunderstanding.

None of this is exotic. What is unusual is treating each as a *mechanical* control with a failing exit code behind it, rather than a practice you hope people follow. With agents, hope does not scale — an agent will always produce a confident answer, so the discipline has to be enforceable rather than cultural.

01 Why ordinary quality practice stops working

Most quality practice rests on an assumption nobody states: that the person writing the test is, at least partly, a second mind. Automation removes that, and several familiar practices quietly become theatre.

PRACTICE	WHY IT DEGRADES
Line coverage targets	Coverage measures *execution*, not verification. A test that calls a function and asserts nothing scores full coverage. Assertion-free tests are an agent's default output when asked to raise coverage — so a coverage target actively rewards the failure mode.
Code review	A reviewer from the same model family shares the author's blind spots. It reads fluently and approves confidently, which is worse than a slow human who is confused and says so.
"Write tests first"	Still valuable, but no longer sufficient. If the agent misread the requirement, the failing test encodes the *same* misreading — so red-then-green proves only internal consistency.
Documented process	An agent reads the document and produces something shaped like compliance. Without a gate, "we follow this process" becomes unfalsifiable.
To-do lists of known gaps	A gap on a list never forces the issue. Nothing fails, so nothing changes. Two years later the document still says the gap is being worked on.

> ◆ THE REFRAMING THAT MAKES THE REST FOLLOW
>
> Stop asking *"is this code correct?"* and start asking **"which of my checks could fail, and have I ever seen one do it?"** A check nobody has watched fail is not a control — it is a comment with a CI badge.

What throughput actually costs

The reason to build this is not caution — it is speed. Agents can produce more change than a team can review by reading. The only way to accept that volume is to make the machine reject the obvious failures, so human attention goes to the judgement calls. Every control here is chosen because it **replaces an act of reading with an act of measurement**.

That is also the honest limit: the harness buys throughput at the cost of building and maintaining the controls. On a small project with one careful author, it is overhead. On a project where agents open a dozen changes a day, it is the only thing standing between you and a codebase nobody can vouch for.

02 Four exit codes, not two

The single highest-leverage idea here, and the cheapest to adopt. It costs one shared constants file and changes how every other control behaves.

EXIT	MEANS	WHY IT NEEDS ITS OWN CODE
0	Verified	The check ran and the property holds.
1	Violated	The check ran and the property is broken.
2	Could not run	Something it needed was missing, so it reached *no verdict*. A tool absent, a credential unset, a service down.
3	Incomplete	It ran, but part of what it should have covered was unreachable — so the property is *unproven* there, not satisfied.

> ◆ WHY TWO AND THREE EXIST
>
> Because **"I checked and it is fine" and "I could not check the important part" look identical from a green badge.** Collapse them into pass/fail and you will eventually ship a control that has never once executed while a dashboard reports it healthy. This is not theoretical — it is the most common way a real quality gate dies, and it dies silently.

The rule that follows, and it needs enforcing in review

Two and three are not passes. Any caller that treats a non-zero code as success has reintroduced the defect the vocabulary exists to remove. Watch for these three shapes:

- `command || true` — swallows every verdict, including real failures.
- `continue-on-error: true` in CI — makes a control advisory. Sometimes correct, but it must be a decision, not a default.
- A tool that "handles missing dependencies gracefully" and returns 0. Graceful is good; returning 0 is not. Return 2.

A worked example of exit 3 earning its place

A project had a rule that its security-critical decision code must never depend on a second language. The check queried the build graph for such dependencies and found none — because the package had no source yet. Reporting **0** would have said "the rule holds". It does not hold; it is *unproven*, because there is nothing to check.

Exit 3 states that difference, and the check stays red until real code arrives. The owner ruled it should stay red rather than be softened — which only became possible once the distinction had a name.

03 Never weaken a check to make it pass

Easy to agree with, hard to hold, and the one rule whose violation is nearly invisible in review — because a loosened gate looks exactly like a gate.

The five ways it happens

MOVE	WHY IT IS THE SAME MISTAKE
Lower a threshold	A floor moved to fit today's number is not a floor.
Narrow a glob	A scan that matches nothing passes trivially, and is indistinguishable from a clean project.
Add a special case for your own file	The exemption outlives the reason and nobody re-litigates it.
Delete an assertion the change broke	The assertion was the control.
Tag an unrelated test so an empty filter is non-empty	Turns a known gap into a false pass — worse than the gap.

What to do instead

1. **Fix the code.** Usually the gate is right.
2. **If the gate is genuinely wrong, that is a finding worth its own change** — with its own reasoning and its own test proving the new behaviour. Fixing a gate *while* making your own change pass is how a weakening gets waved through.
3. **If neither, record a decision** (§5) and route around it.

> ✗ **TWO PHRASES THAT SHOULD STOP A REVIEW**
>
> *"This check is too strict for now"* and *"I'll tighten it again later."* Both describe permanently loosening a control while implying it is temporary. If it really is temporary, it needs a recorded decision with the condition that reverses it — otherwise "later" never arrives, and nobody remembers the check was ever stricter.

Make the rule visible where it is broken

Put it in the failure message, not only in a contributing guide. Every gate in the reference implementation ends its refusal with a version of: *fix the code; do not weaken this check; if the check is wrong, that is its own change.* The person about to take the shortcut is reading the error, not the handbook.

04 Every gate ships a test that proves it can fail

If you adopt one thing from this document, adopt this. It is the control that keeps every other control honest, and it is usually missing.

> ◆ THE CORE INSIGHT
>
> **A gate that always passes is indistinguishable from a codebase that is always correct.** You cannot tell them apart from the output, because the output is identical. A typo in a glob, a swallowed exception, an inverted condition — any of them produce a check that is green forever, and nothing about a green check invites suspicion.

What a gate fault suite does

For each gate, in a throwaway copy of a repository, it **makes the violation real** — creates the malformed row, deletes the required field, edits the shared file, commits a fake credential — and asserts the gate's **exact exit code**.

REQUIREMENT	WHY
Make the violation real	A mocked violation proves the mock. Create the actual condition on disk.
Assert the exact code	1 and 3 are different claims. A suite accepting "any failure" passes while a gate conflates them — the confusion the codes exist to prevent.
Include negative cases	A gate that fires on everything gets disabled within a week. Prove it stays quiet when it should.
Run it even when a gate is red	When a gate is failing, *"is the gate still sound?"* is precisely the question worth answering. In CI, guard it so an earlier failure cannot skip it.

It pays for itself immediately

In the reference implementation the suite found **seven real defects in the gates**, every one of them a silent pass or a mis-report, and none visible by reading the code:

DEFECT	WHY IT WAS INVISIBLE
A gate scanned source for references to decision ids — **including its own documentation**, which used a real id as an example	It flagged itself. The fix produced a convention worth keeping: prose that *discusses* an id must write it unnumbered.
The example row inside a fenced code block in the gate's own **template** parsed as a real row	Every fresh adopter would start with a phantom decision, and hit a duplicate-id error the day they added a real one.
Only ``` fences were stripped, not `~~~`	Markdown allows both. A template using tildes reintroduced the bug above.
The "what differs in the code" check accepted `` `.` `` — a single backticked dot	The most important field in the register could be satisfied by a gesture.
The source scan allow-listed four directory names, so code in `internal/` was **invisible**	A silent pass on a repository full of code, from the gate whose purpose is catching silent passes. Fixed by inverting to a deny-list by extension.
That inversion then counted the harness's **own scripts** as product source	The harness reported that the harness was unmeasured, forever. Fixing one direction of a scan opened the other.

DEFECT	WHY IT WAS INVISIBLE
A **crashing** gate exited 1, so the runner reported "the property is broken"	It sends someone hunting for a defect in the code when the defect is in the gate. A crash is a could-not-run.

Note the shape of the fifth and sixth together: **fixing a silent pass in one direction created one in the other.** That is the strongest possible argument for the suite — not that it catches mistakes, but that it catches the mistakes your fixes introduce.

05 The decision register — stop, don't guess

> ◆ IN ONE PARAGRAPH
>
> When work hits a question the design does not settle, the answer must be **stop and record it**, not **pick the sensible option**. A register holds those questions as numbered rows with the options written down, and work routes around them. It matters more with agents than with people, because an agent will *always* produce a confident answer — that is what it is for.

The cost of guessing, stated precisely

Once a guess is in the code it is indistinguishable from a decision. Six months on nobody can tell which lines reflect a deliberate choice and which reflect somebody's Tuesday afternoon. Changing them becomes frightening, because you cannot tell what you might be undoing. The register's whole job is keeping those two categories apart.

The fields, and why each is required

FIELD	WHY
Status	Open or resolved. Nothing else — a third value means the register has drifted into free text and a reader can no longer tell blocked from settled at a glance.
Issue link	**The delivery mechanism.** A row in a file nobody opens has told nobody anything. The tracker's assignment notification is what reaches a human. Most often missing; most expensive when it is.
Candidate answers	The options, labelled. A question is not an answer — a decider must see what they are choosing between without reconstructing it.
What differs in the code	**The entry condition**, and the cleverest rule here. See below.
Ruling	Empty until decided, then a durable, findable reference. "We agreed in a meeting" cannot close a row, because a later reader cannot look it up.

> ◆ WHY "WHAT DIFFERS IN THE CODE" IS THE LOAD-BEARING FIELD
>
> Name what would be *physically different* in the codebase under each option. If you cannot, **this is a task, not a decision** — and it does not belong in a decision register.
>
> Without this test the register slowly fills with to-do items, and then nobody reads it, and then the one genuinely blocked decision in it is invisible. Gate the field and a task dressed as a decision is rejected automatically.

Two rules that seem pedantic and are not

- **A number is never reused and never renumbered.** A row keeps its id for life and only its status changes. Renumbering on closure dangles every citation of the old number and frees the number for a different question.
- **Never cite another repository's number.** These numbers are per-repository and they collide. In the reference implementation two repositories each had a `−7` that turned out to be the *same* question recorded twice with divergent state, while another's `−21` had no counterpart at all. Cite by **subject and repository**.

The citation scan

The gate also checks the reverse direction: if any document cites a decision id, a row with that id must exist. **A decision cited but never recorded is the precise failure the register exists to stop** — it is how a question gets discussed,

referenced, and never actually answered.

06 Trigger gates — catching "not applicable" as it expires

The subtlest control here, and the one that most often does not exist anywhere. It turns a to-do item into something that fails on its own schedule.

The pattern it kills

Someone writes "we should add mutation testing" in a document. It becomes an item on a list. The list is reviewed occasionally and the item is still true, so it stays. **Nothing ever fails, so nothing ever forces the issue.** Two years later the document still claims mutation testing is the coverage metric and no package has ever been measured.

The shape of a trigger gate

Distinguish two states that otherwise both look fine:

STATE	VERDICT	WHY
Not applicable	✓ PASSES	Nothing to measure — but the gate **states its trigger**, because this stops being true silently.
Unobtainable	✗ FAILS	There *is* something to measure and the measurement cannot be produced. A defect.

The second is the obvious value. **The first is the one that matters more**, because that transition — the day someone adds the first source file to a package that had none — is exactly what nobody is watching for.

Verified as a state machine, not a run

A trigger gate is only worth having if the trigger fires. Test it by moving through the states:

```
no source            → exit 0, and the message names the trigger
add one source file  → exit 1
remove it            → exit 0
a TEST file only     → exit 0   (mutating a test is meaningless)
```

Where else the shape applies

- **Accessibility checks** — not applicable until the first user-facing surface exists; red the moment one does.
- **Load tests** — not applicable until something serves traffic.
- **Migration safety** — not applicable until there is a schema.
- **Licence scanning** — not applicable until there are third-party dependencies.

In each case the ordinary approach is a checklist item that nobody actions. The trigger gate turns it into a check that begins failing precisely when it starts mattering.

07 Drift protection for shared files

Once you have more than one repository, some files want to be shared. The clean answer is to publish them as a versioned package. Teams rarely start there, so the files get copied — and then they rot.

How the rot actually happens

Someone improves the original. The copies keep the old behaviour. Nothing anywhere reports it. Measured in the reference implementation: an upstream change added four lines to a shared review configuration, and **three downstream copies went stale the same day** — in the files that decide how code gets reviewed.

Drift protection does not fix that. It makes it **visible**, which is what you can have before the publishing question is settled.

The mechanism

PIECE	WHAT IT DOES
A manifest	For each copied file: which upstream revision it came from, and its content hash.
A gate	Recomputes each hash and compares. A mismatch means someone edited the copy instead of the original.
Two file classes	**Verbatim** — byte-identical, checked by hash. **Templated** — differs by a known substitution, so instead the check confirms the substitution is *complete*.

> ◆ KEEP COPIES BYTE-IDENTICAL, EVEN WHEN A FEW LINES DO NOT APPLY
>
> A file adapted for local taste is a file nobody can diff, and it forks quietly. In the reference implementation four shared files were copied **in full** rather than trimmed, though five lines out of 131 referenced things one repository did not have. Forking a file to adjust 4% of it trades a small correctness gain for undetectable drift. Local context belongs in a document, never in the file.

> ✗ STATE THE LIMIT IN THE GATE'S OWN OUTPUT
>
> A pass means **unchanged since copied**. It does **not** mean **up to date**. Detecting that upstream has moved ahead requires reading upstream's tree — network access and a credential — and a well-structured upstream cannot push the check either, because it does not depend on its consumers.
>
> So **the direction that causes real staleness stays undetected**. Say that where the result is read. A green check that gets over-read is worse than no check.

Treat the copying itself as a recorded decision

Hand-copying is a workaround, and workarounds become permanent when nobody names them. Put the publication question in the decision register with its options — publish a package, or keep copying with a drift gate and a staleness check — and what differs in the code under each. Otherwise you have chosen by default, and nobody noticed choosing.

08 Making CI tell the truth

Two properties of CI itself, both easy to violate by accident, both silent when violated. Assert them over the workflow files with a test, because neither is visible from a badge.

Law 1 — no verdict may be hidden by an earlier failure

Most CI systems skip the remaining steps of a job as soon as one fails. So if a job runs two *independent* checks and the first fails, the second is reported as *skipped* — and a reader cannot tell a check that passed from one that never ran.

Where two independent verdicts share a job, every step after the first needs a failure-surviving condition. A step that is a genuine *prerequisite* of the next is exempt: its failure stopping the job is correct, because the next step could not have run meaningfully anyway.

> ◆ THIS IS NOT HYPOTHETICAL
>
> When this test was ported into three sibling repositories it found **six violations in each one** — including a case where a gate had been made deliberately red, which meant it was silently skipping **its own fault-injection suite**. The check that proved the gate still worked had stopped running, and nothing said so.

Law 2 — a blocking check's tools must be installed in its own job

If a check shells out to a tool, the job running it must install that tool. Handling absence gracefully is not enough: a check that exits "could not run" is honest and still means the property went unverified.

The reference implementation had a check that **had never once executed in CI** — no job installed the package manager it needed, so it died on a missing binary, failed first, and hid a second check behind it. It was red for an environment reason, never for the property it checked.

One verdict per job, where you can afford it

Splitting independent checks into separate jobs makes Law 1 impossible to violate and each verdict individually readable. It costs job start-up time, so it is a trade rather than a rule — but it is the right default for anything expected to be red for a known reason, so its red cannot mask anything else.

Beware the friendly idioms

IDIOM	WHAT IT ACTUALLY DOES
`|| true`	Discards every verdict, including real failures.
`continue-on-error`	Stops a step failing the job — and does nothing about that step being *skipped* by an earlier failure. It does not satisfy Law 1.
A tag filter matching no tests	Many runners exit successfully on "no tests found". A documented command that matches nothing therefore reads as a pass while testing nothing.

09 Independence controls that outrank tests

Four controls whose verdict does not depend on whoever wrote the code. Adopt in this order — each is roughly an order of magnitude more work than the one before.

CONTROL	EFFORT	WHAT IT REMOVES
Mutation testing	low	*"Who tests the tests?"* It breaks your code deliberately and checks whether any test **notices**. A test asserting nothing kills no mutants. **Use this instead of a coverage target, not alongside one.**
Property tests	low–medium	*"Try the inputs I would never think of."* You state a **law** rather than an example, and the tool generates hundreds of cases, then shrinks any failure to the smallest one that still breaks.
A different-lineage reviewer	medium	A reviewer from a different model family does not share the author's blind spots. In the reference implementation the first real use of one found a **factually wrong comment in the very change that introduced it** — a claim about a tool's defaults that the tool's own schema contradicted.
Blind conformance tests	high	A second agent writes tests **from the requirement alone**, never reading the implementation. Hand it a signatures-only file so it physically cannot copy your logic.

> ◆ THE RULE THAT MAKES BLIND CONFORMANCE WORK
>
> **If the two disagree, that is a finding for a human.** The implementer does not resolve it — their reading of the requirement is precisely what is in question. Let the implementer adjudicate and you have rebuilt the single-mind problem with extra steps.

Quote a measurement with its denominator

A mutation score of 99% sounds conclusive and can be nearly meaningless. In the reference implementation the real figure was **99.36% over 472 mutants that compiled** — with a further 652 of 1,124 excluded because they did not compile at all.

That exclusion is correct: a mutant that cannot compile is not a fault a test could catch, since the compiler rejects it in production too. But **"99.36%" without the denominator invites a conclusion the number does not support.** Make your tooling refuse to emit a score at all when everything was excluded, so an empty run cannot pass as a good one.

What to do when a control does not exist yet

Say so, in the place someone would look for its result. The reference implementation documented a fault-injection command in three places; it matched zero tests and exited as success. The honest statement — *"no fail-closed behaviour here has been proven to fail closed"* — is more useful than a command that looks like proof. And never make an empty filter non-empty by tagging an unrelated test: that converts a known gap into a false pass.

10 Grounding, and writing it back

Two process controls rather than tools. They are what stop an agent inventing requirements, and what stops the reasoning behind a change evaporating the moment it merges.

Grounding — quote the requirement, do not paraphrase it

Before writing code, find the written requirement that governs it and **quote it exactly** in the change. This sounds bureaucratic and is not: a paraphrase is where the misreading enters, and a paraphrase in a commit message is indistinguishable from the real thing six months later.

- **Keep requirements addressable.** Give settled decisions short identifiers so a change can cite one precisely.
- **Check for supersession.** The highest-risk failure is building correctly against a rule that a later decision overrode — it produces confidently wrong code with a citation attached.
- **An index is a router, not a source.** If you generate a summary over your requirements, never implement from the summary. Open the thing it points at.
- **No requirement means halt**, not "use judgement" (§5).

Writing it back — the implementation record

A change is not done when it merges; it is done when someone else can understand why it looks the way it does. One record per change, covering:

SECTION	CONTENT
Grounding	Each requirement, quoted. Which supersessions you checked. An explicit statement that you made no assumptions.
Exact implementation	Files and decisions, precise enough that the next person does not re-derive them. Include approaches you tried and rejected — that is the most expensive knowledge to reconstruct.
Test proofs	**What each control proves** — not that it passed. And if a control was not run, **say so here**.
Not done	Deliberate gaps and follow-ups, so the next reader knows what they are looking at.

> ✗ **THE FAILURE THIS SECTION EXISTS TO PREVENT**
>
> A record that lists the tools which ran is not evidence — it is a receipt. *"Mutation testing: passed"* tells a later reader nothing they can act on. *"Mutation 99.36% over 472 compilable mutants, so a test noticed almost every change we could make"* does.
>
> And a record claiming a control that never ran is worse than one admitting a gap, because somebody will rely on it. **Declaring what you did not do is the part that makes the rest believable.**

11 What this pattern cannot give you

Stated plainly, because a pattern document that only lists benefits is the same genre of dishonesty the harness is built to prevent.

LIMIT	WHAT IT MEANS IN PRACTICE
None of it blocks anything without branch protection	**The most important line here.** Gates, hooks and review panels all only advise. Until your host is configured to require the checks and forbid bypass, a determined or hurried author merges past every one. In the reference implementation, before protection was configured: **134 of 134 changes merged by their own author, zero approvals ever recorded, median 39 seconds from open to merge.** The harness was excellent and enforced nothing.
A hook is a convenience, not a control	Anyone can skip it. It catches the honest mistake cheaply. It stops nothing determined.
Drift protection is one-directional	It catches local edits, not upstream moving ahead — the direction that actually causes staleness (§7).
It cannot tell you the requirement was right	Every control here checks fidelity to a written requirement. If the requirement is wrong, the harness will help you implement it wrongly with great rigour. That judgement stays human.
It has a maintenance cost	Gates need their own tests, and those tests need maintaining. On a small project with one careful author this is overhead. It pays off in proportion to how much change you accept without reading all of it.
Volume of green is not maturity	Fifteen passing tests on a mostly-scaffold project is a low bar. Documents resolving is link-checking, not capability. Read the list of what does *not* work; that is the honest picture.

Adopt in this order

1. **Branch protection.** Free, immediate, and without it nothing else binds.
2. **Four exit codes.** One small file; changes how everything else behaves.
3. **One gate plus its fault suite.** Whichever rule you most fear breaking. Watch it fail once.
4. **The decision register.** Cheap, and it starts capturing value the first time someone would otherwise have guessed.
5. **Mutation testing on one package**, replacing any coverage target.
6. **Trigger gates** for each control you know you will need later.
7. **A different-lineage reviewer**, then drift protection when a second repository appears.

Steps 1 and 2 take an afternoon and remove the two failure modes that matter most: nothing binding, and a check that never ran reading as a pass.

The Agent Development Harness — Pattern Handbook. A reusable pattern, extracted from a production implementation and stripped of anything project-specific. Every claim about a failure mode in this document is something that happened, not something imagined. Companion document: **Adopting the Harness**, which is the same material as a setup sequence with working code.